

HospiCart System

Design Document



Databases 2025

Abril Maitena Betti, Lucía Fandiño and Marta Guzmán



Index

1. Overview
2. Main Goals and Motivation
3. Architecture and Package Structure
 - 3.1 HospiCartPOJOs
 - 3.2 HospiCartInterfaces
 - 3.3 HospiCartJDBC
 - 3.4 HospiCartJPA
 - 3.5 HospiCartApp
 - 3.6 Exceptions
 - 3.7 HospiCartXML
 - 3.8 Utilities
4. Conclusion about Design Decisions
5. Use Case Diagram, ER Diagram, UML and Mockup



1. Overview

HospiCart is a Java-based medical product ordering system, inspired by the Amazon shopping experience but specifically tailored for hospitals and healthcare professionals. Therefore, our platform sells sanitary products of different types (such as medicines, disposable supplies used by professionals, machinery, etc.) Furthermore, this application enables doctors and nurses to order clinical and surgical supplies, while suppliers manage product's inventory and orders made by clients (nurses or doctors).

In addition, the platform is connected to a database using JDBC and JPA, in which all the information regarding our clients, products, orders and suppliers is safely stored.

2. Main Goals and Motivation

- Provide a centralized platform for purchasing and managing healthcare products.
- Enable healthcare professionals (doctors and nurses) to efficiently browse, select, and order clinical supplies.
- Allow suppliers to register their products and manage availability through a simple interface.
- Reduce manual procurement processes and promote digital ordering in hospitals and clinics.
- Ensure a user-friendly experience adapted to healthcare workflows.

3. Architecture and Package Structure

The HospiCart system follows a layered architecture, separating responsibilities across different packages to improve maintainability, testability, and scalability. Each package represents a different aspect of the application's logic or technical role, from the data model to the user interface and utilities.

3.1. HospiCartPOJOs

These classes represent real-world entities within the system. Therefore, they contain the necessary fields that define the entities in our program, alongside several methods (such as getters, setters, equals, hashCode and to string) which are not only useful but required for POJOs.

We have defined seven main domain entities based on the ER model:

1. **Client:** Represents the end users of the system (doctors and nurses). Each client has personal data (name, phone, email, address) and is associated with multiple orders.
2. **Order:** Captures the full purchase request made by a client. It includes a status (e.g., pending, shipped), date, and is linked to one shipment and one payment. Orders are composed of one or more products.
3. **Product:** Represents a medical item available for purchase. Each product includes metadata such as category, price, stock, and whether it requires a prescription. Products are offered by suppliers.
4. **Supplier:** Suppliers are responsible for providing products to the platform. They manage product details, stock, and contact information.
5. **ProductOrder:** This is a relational entity (join table) that resolves the many-to-many relationship between Order and Product. It includes the quantity of each product in a specific order and the subtotal price.
6. **Payment:** Stores the payment amount, method, and status
7. **Shipment:** Contains tracking data for the delivery.

All the object relationships are detailed visually in the UML class diagram and the ER diagram (see Section 5).

3.2. HospiCartInterfaces

This package defines the **service layer abstractions** using Java interfaces. Each manager class interface defines operations relevant to a specific part of the application, such as:

- IUserManager – handles user registration, authentication, and role management.
- IOrderManager – manages order creation, update, deletion and retrieval.
- IProductManager, ISupplierManager, etc. – manage corresponding entities regarding their creation and insertion into the database, update, removal and retrieval.

These interfaces are designed to make the system flexible. They allow different implementations (like JDBC or JPA) to be used as needed, without having to change the main program logic or user interface.

3.3. HospiCartJDBC

These classes are responsible for working directly with the database using SQL queries and prepared statements. We chose JDBC because it gives us more control over how the database works, which is useful for learning and for building simple project prototypes where using a full ORM like JPA might be too much.

These classes implement the interfaces explained above, which enabled us to arrange our application's functionality and identify the responsibility of each of its classes in a clearer way.

Also, all the database connections are managed through the ConnectionManagerJDBC class. This helps us reuse connections, avoid repetition, and handle errors more easily.

3.4. HospiCartJPA

Regarding the JPA package, it includes the class UserManagerJPA, which handles all user-related operations—such as registration, login, role assignment, and password (which is encrypted) updates—using the Java Persistence API (JPA) instead of JDBC.



This class uses an `EntityManager` to connect to the database and manage transactions. It includes native queries for tasks like retrieving users by email, while also benefiting from JPA's ability to handle objects and relationships more cleanly. It also ensures that foreign key constraints are active and that roles (doctor, nurse and supplier) are created when the system starts.

3.5. HospiCartApp

The “HospiCartApp” contains a general main, which handles user registration and log in and then is in charge of redirecting the user towards the main menu that applies depending on the role of the user. Therefore, the package also contains one main for each role the application admits: doctors, nurses, and suppliers.

The key components include:

- **MainMenu:** Displays the initial menu where users can register, log in, or reset their password. Depending on the selected role (doctor, nurse, or supplier), it redirects the user to the appropriate role-specific menu. Upon launching the application, products are loaded along with their respective fields and associated suppliers. This initialization uses `product.txt` and `supplier.txt` located in the utilities package.
- **DoctorMenu:** provides a session for doctors to browse products, manage their cart (which is implemented as an “Order”), pay, and update personal information. It connects to various managers (e.g., “`ProductManagerJDBC`”, “`OrderManagerJDBC`”) through a shared “`ConnectionManagerJDBC`” object.
- **NurseMenu:** similar to the doctor menu, but a little bit restricted in functionality. The difference between these two types of menus relies on the fact that doctors have prescriptions whereas nurses do not. Therefore, while doctors can access every product our application sells and purchase them, nurses “shopping catalogue” is restricted to those products that do not require prescription in order to be bought. Furthermore, and excluding the aforementioned difference, the functionality of the nurse menu coincides with that of the doctor menu.
- **SupplierMenu:** allows suppliers to view and manage their company's product listings, restock items, and view customer orders by status. Suppliers can update



product information, add new products, and monitor stock levels. They are linked to a company name and load their product catalog upon login.

The menus rely on keyboard input/output, implemented through utility classes (“InputKB”, “Output”) for consistent formatting and handling.

In conclusion, this package connects the user actions to the corresponding logic and database operations through manager classes, using menus adapted to each role.

3.6. Exceptions

This package defines personalized exception classes that capture unexpected situations the program may face, such as:

- Invalid user actions.
- Database errors.
- Failed transactions.

Using custom exceptions improves code clarity and allows for precise error handling without relying on generic “SQLException” or “RuntimeException”.

- 3.7 HospiCartXML
- 3.7 Utilities

3.7. HospiCartXML

This package includes the manager class that implements XML, which has useful methods such as the marshaller and unmarshaller, and a second class which marshals and unmarshals dates. In addition, this package contains the DTD documents we generated for both “Client” and “Product”.

3.8. Utilities

This package contains the class that is in charge of encrypting the password of the users, classes that handle the input and output interaction with the user (through several methods, such as reading methods).

4. Conclusion about Design Decisions

In order to conclude, we will summarize the main decisions that shaped the development of “HospiCart”, explaining why each one was taken.

- **Use of Java:** we chose Java because it's widely used, reliable, and supports good object-oriented programming practices. It also works well with tools like JDBC and JPA.
- **Layered Structure:** we organized the system into layers (POJOs, interfaces, application logic, and exceptions) to separate responsibilities. This makes the code easier to manage, test, and improve in the future.
- **Mix of JDBC and JPA:** We used JDBC for most of the system because it gives us full control over SQL queries. For the user and role management, we used JPA to simplify object-relational mapping.
- **Use of Interfaces:** by defining interfaces (like “IUserManager” or “IProductManager”), we made it easier to switch between JDBC and JPA and to add new features later without changing the rest of the system.
- **Console Interface:** we built a text-based user interface for simplicity and learning purposes.
- **Entity Relationships:** we created clear relationships between entities based on the ER diagram. For example, “ProductOrder” was added as many to many relationships between “Product” and “Order” with attributes, so we can store important data like quantity and subtotal for each product in an order.
- **Menus by Role:** each user type (doctor, nurse and supplier) has a specific menu adapted to their needs. This improves usability and avoids errors—for example, nurses can't order items that require prescription.
- **Custom Exceptions:** we created our own exception classes to handle errors more clearly and specifically.

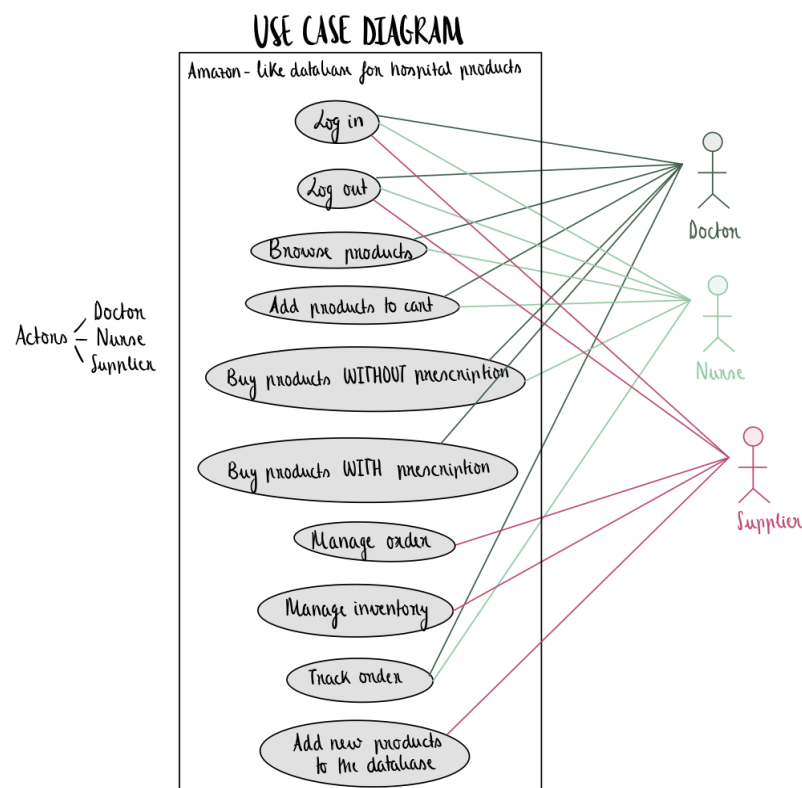
5. Use Case Diagram, ER Diagram, UML, Mockup and Traceability Matrix

The following diagrams and interface prototypes provide essential visual documentation that supports and validates the design choices made for the “HospiCart” system.

- **Use Case Diagram**

The use case diagram illustrates the main interactions between system actors (doctors, nurses and suppliers) and the functionalities provided by the “HospiCart” platform. It includes essential use cases such as:

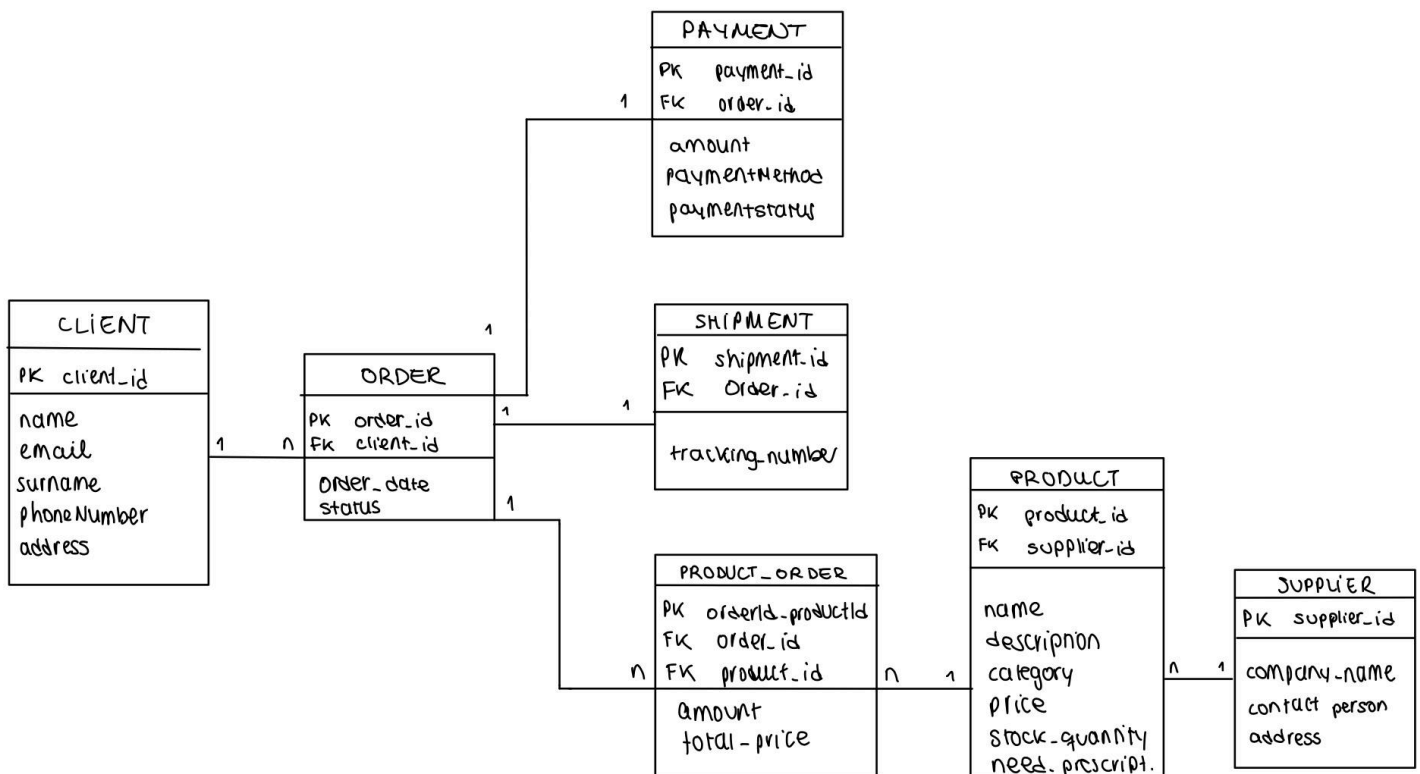
- Searching for medical products
- Viewing product details
- Adding products to the cart
- Placing an order
- Managing inventory (for suppliers)



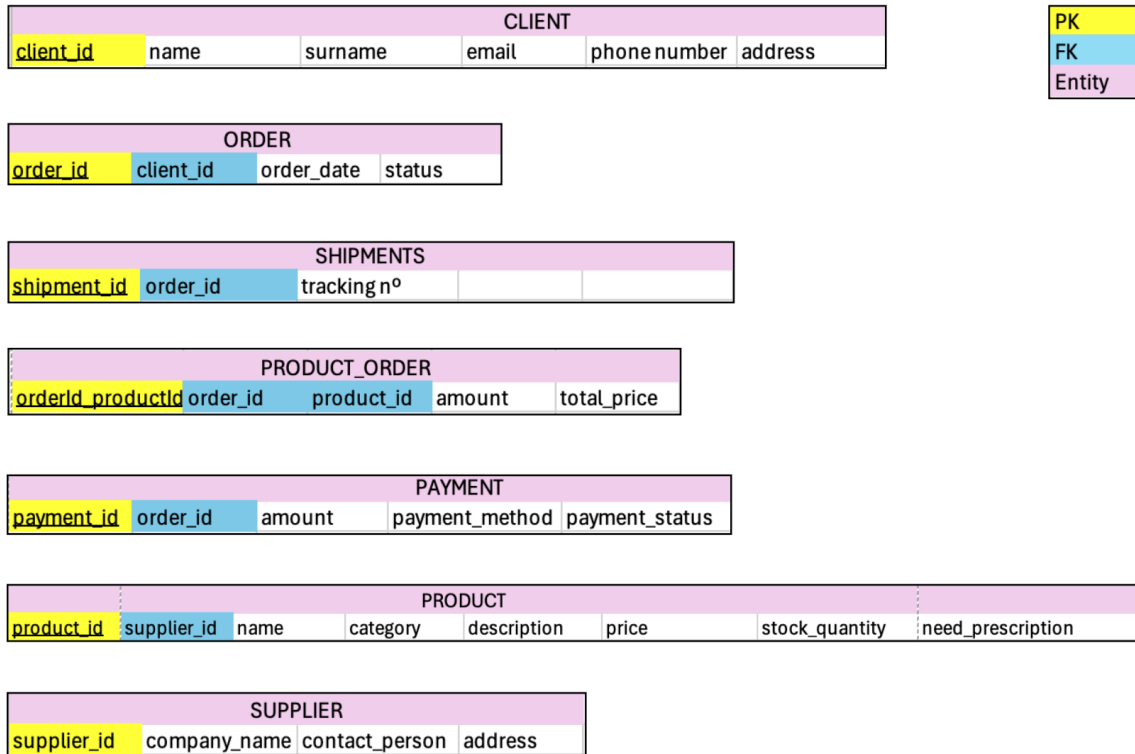
- **Entity-Relationship (ER) Diagram**

The ER diagram defines the logical data structure of the “HospiCart” database. It includes all the entities that built up our program, alongside their attributes, primary keys, foreign keys and relations with other entities.

As explained before, an important consideration about our ER diagram and the general structure of our program, is that what is depicted as an entity called “ProductOrder” is, in reality, a many to many relationship between “Product” and “Order” with attributes.



- **Tables**



- **UML Class Diagram**

The UML class diagram reflects the object-oriented design of the system. It shows the main classes, their attributes, methods, and relationships. This helps bridge the gap between the conceptual design and the implementation in Java.

The following images depict the UML we developed for our project.

Hospital App

<<class>> MainMenu

- + main(args: String[]): void
- + setApplication(): void
- + register(): void
- + registerDoctor(): void
- + registerNurse(): void
- + registerSupplier(): void
- + login(): void
- + resetPassword(): void

<<class>> DoctorMenu

- + DoctorMenu(cm: ConnectionManagerJDBC, doc: Client)
- + displayDoctorMenu(): void
- + productShoppingMenu(chosenProduct: Product): void
- + viewCart(): void
- + addToCart(product: Product, amount: int): void
- + browseProductsByCategory(category: Category): void
- + pay(): void
- + accountSettings(): void
- + client2Xml(): void

<<class>> NurseMenu

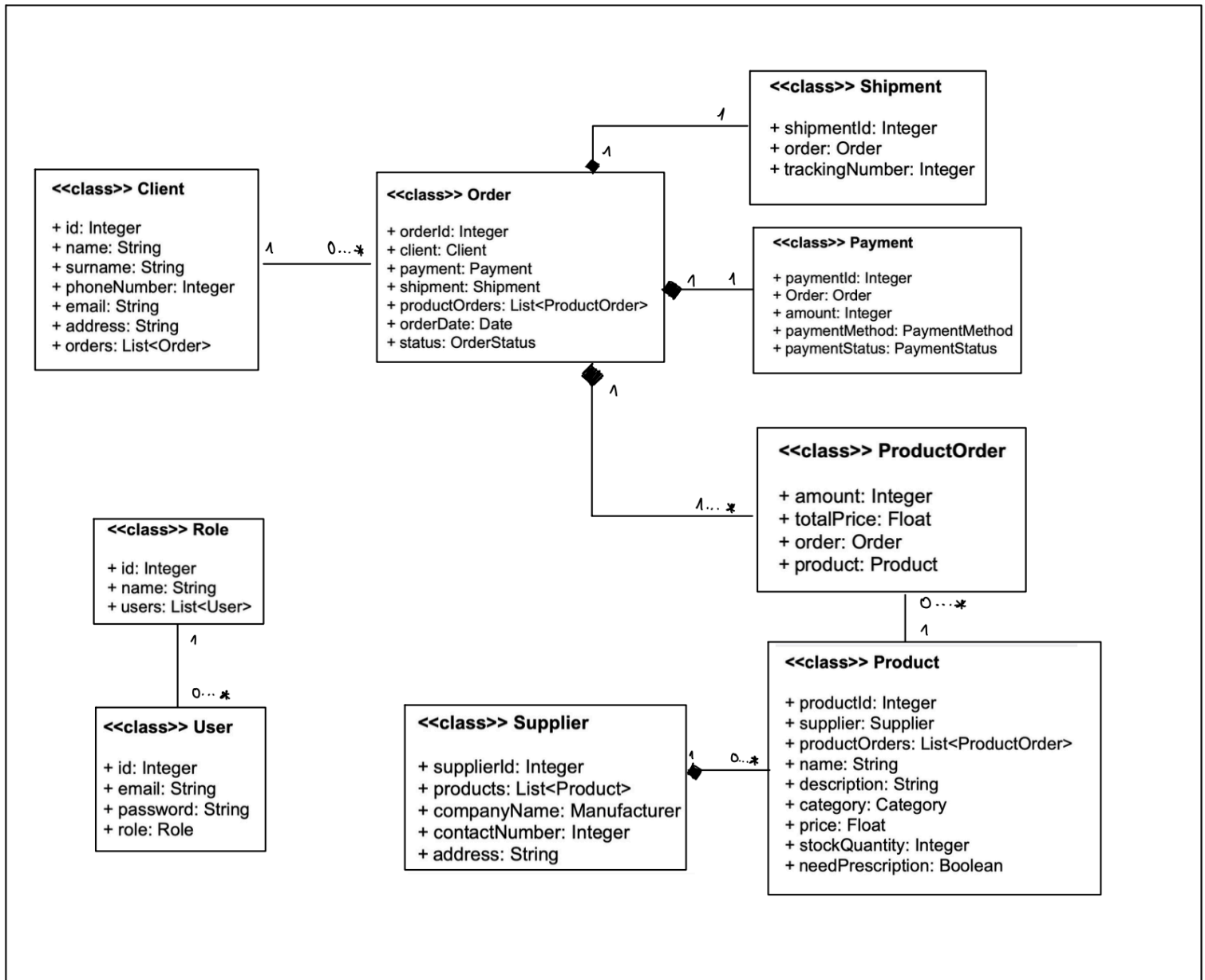
- + NurseMenu(cm: ConnectionManagerJDBC, nurse: Client)
- + displayNurseMenu(): void
- + productShoppingMenu(chosenProduct: Product): void
- + viewCart(): void
- + addToCart(product: Product, amount: int): void
- + browseProductsByCategory(category: Category): void
- + pay(): void
- + accountSettings(): void
- + client2Xml(): void

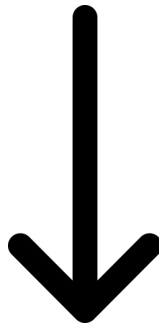
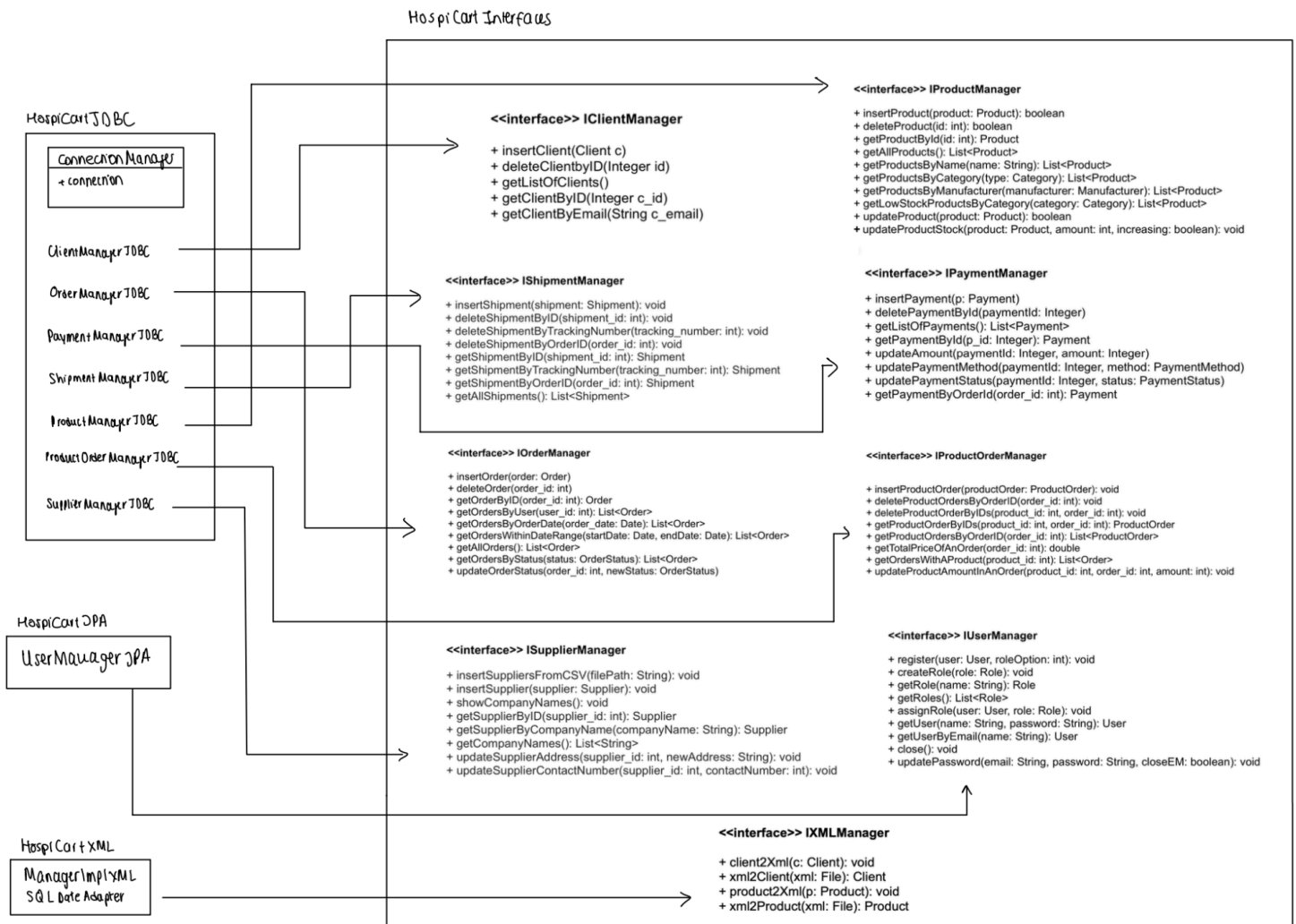
<<class>> SupplierMenu

- + SupplierMenu(cm: ConnectionManagerJDBC, supplierUser: User)
- + displaySupplierMenu(): void
- + displaySupplierMenuOptions(): void
- + approveOrder(order: Order): boolean

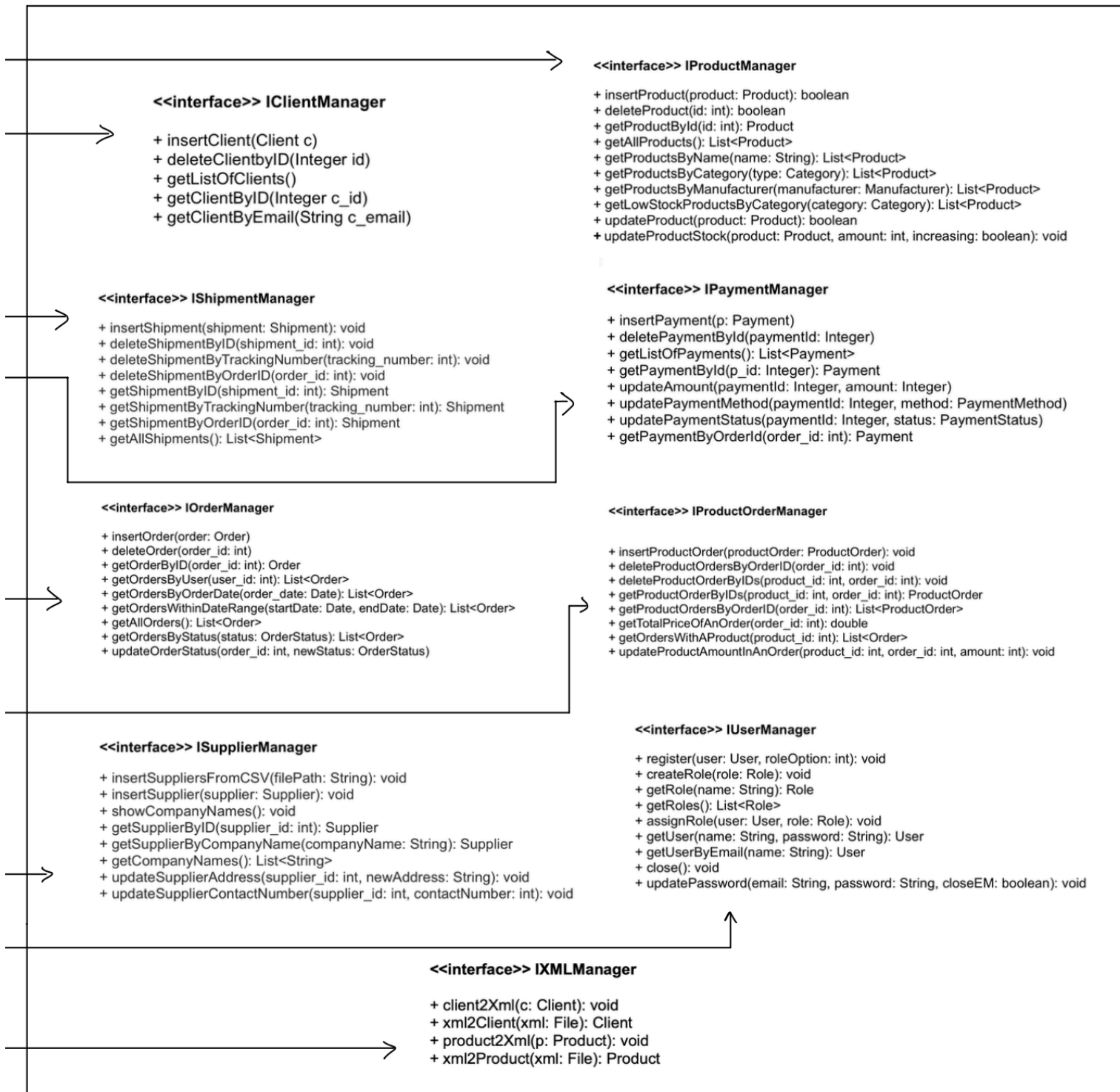
Exceptions

ClientException
OrderException





Hospital Interfaces



- **Traceability matrix**

Regarding the traceability matrix, which is visible below, we built it up taking into consideration how all the system requirements should connect with their corresponding functionalities (use cases). By doing this, we ensured that all specified needs (such as user registration, logging in, managing inventory, and purchasing products) were properly implemented in the system.

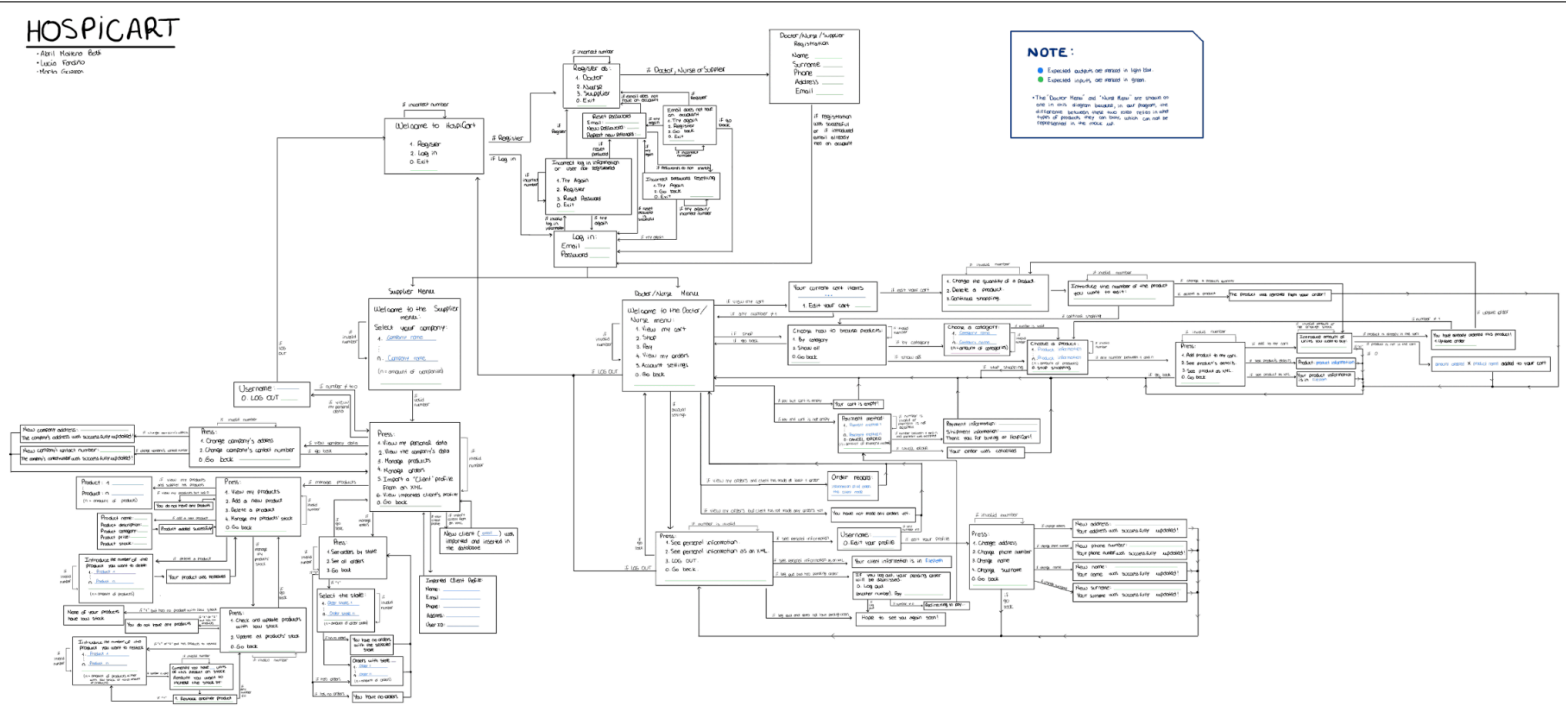
HOSPICART	Req - The introduced email does not have an account	Req - The user has a registered account	Req - The user has logged in	Req - The username is valid	Req - The password is valid	Req - The user has prescription	Req - The cart is not empty	Req - The payment is accepted	Req - A tracking number is generated	Req - The user has at least one order associated to his account	Req - The user is not trying to add a repeated product	Req - Products with no stock can not be bought	Req - Restock products
	UC-1 Register	✓		✓	✓								
	UC-2 Log In		✓	✓	✓								
	UC-3 Log out		✓	✓									
	UC-4 Search products		✓	✓									
	UC-5 Add products to cart		✓	✓								✓	
	UC-6 Make purchase with prescription		✓	✓		✓	✓	✓	✓				
	UC-7 Make purchase without prescription		✓	✓			✓	✓	✓				
	UC-8 Track order		✓	✓						✓			
	UC-9 Manage inventory		✓	✓									✓
	UC-10 Register products		✓	✓							✓		

- **Mockup diagram**

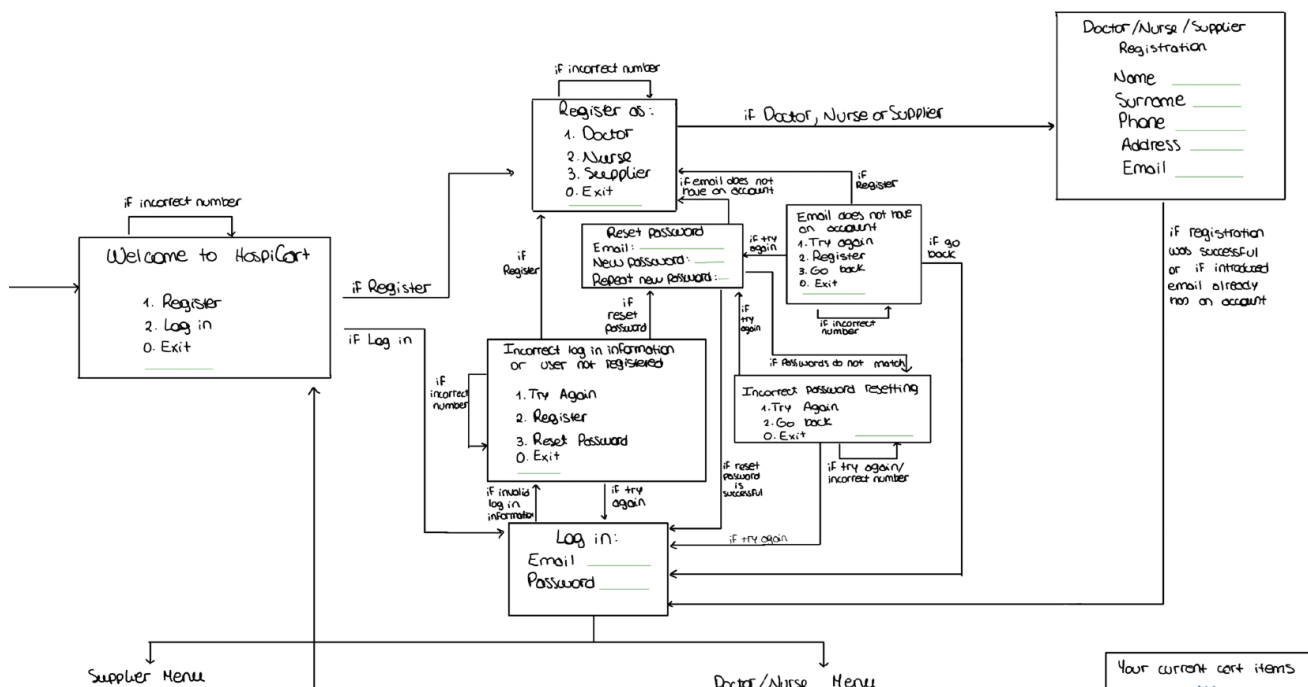
In respect of the mockup, it visually represents the main interface and user interactions within the system. It outlines the layout and design of key screens such as login, product browsing, shopping cart, order tracking, and inventory management. The mockup is tailored for two main user types: healthcare professionals (doctors and nurses) and

suppliers. It provides a user-friendly and intuitive navigation flow, helping to illustrate how users interact with the platform to perform specific actions.

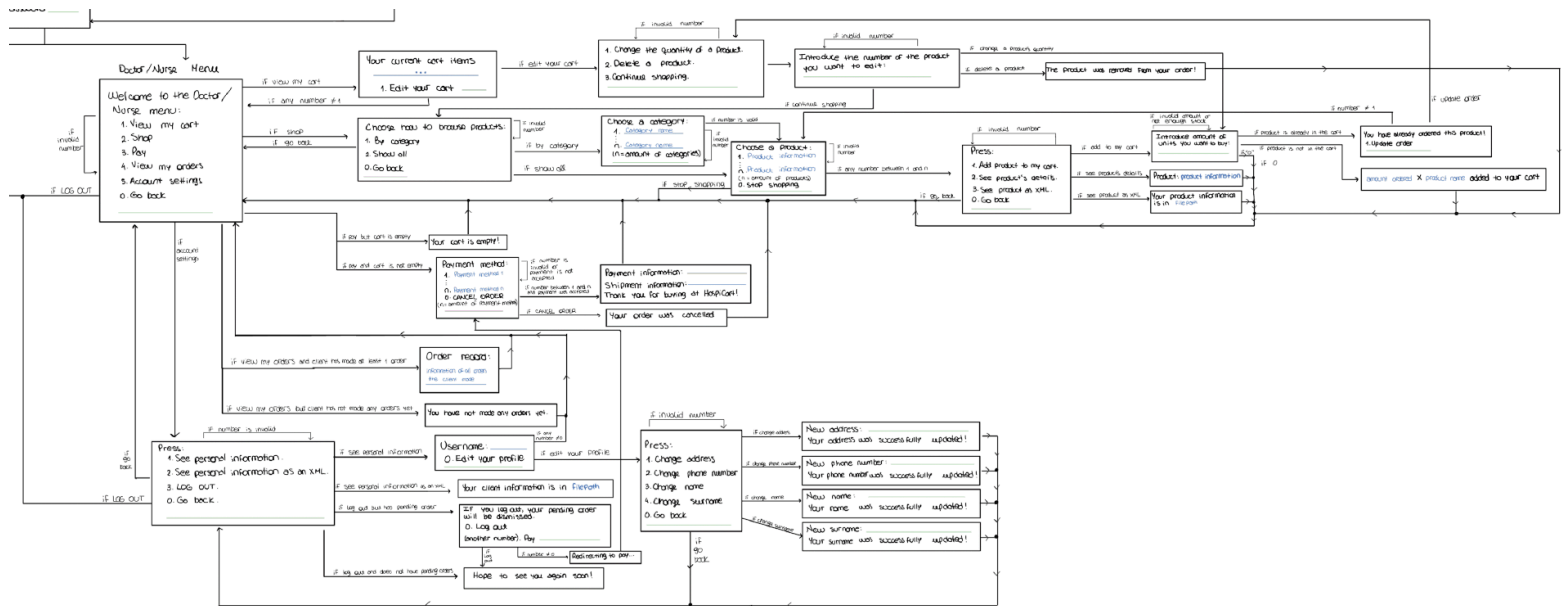
The following image provides a general view of the mockup we developed for “HospiCart”. As we can see, the mockup is very extensive, which is why we will provide more images below in order to take closer looks at the mockup parts.



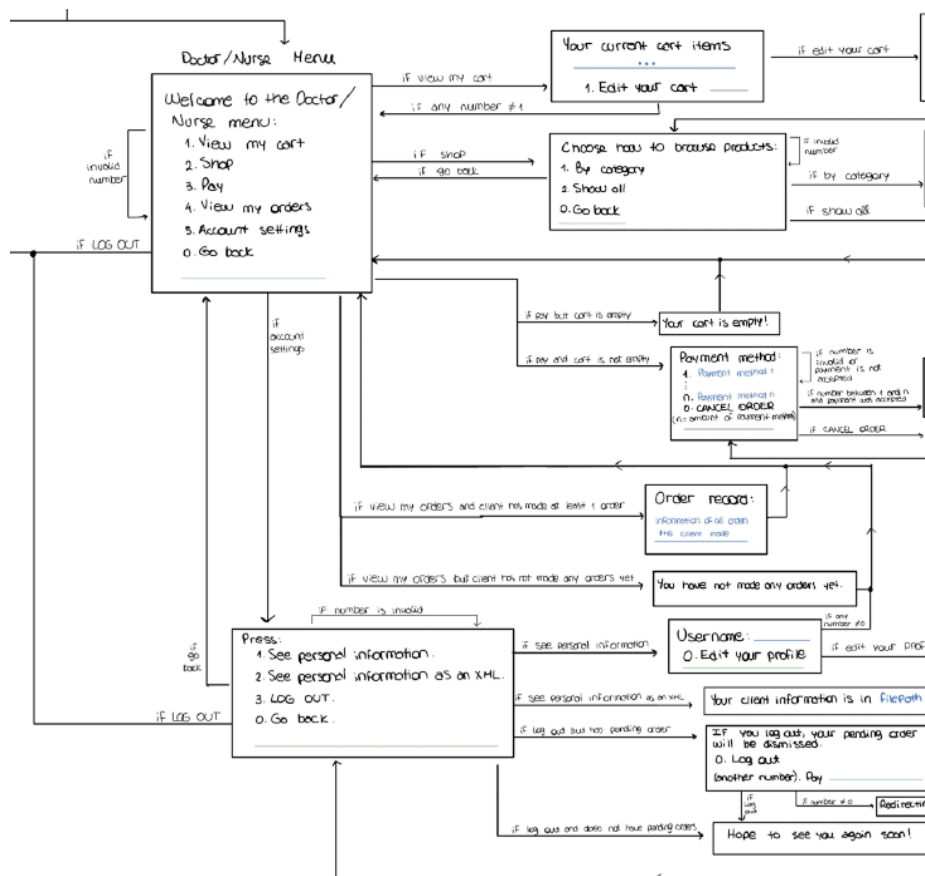
Login and Register logic:

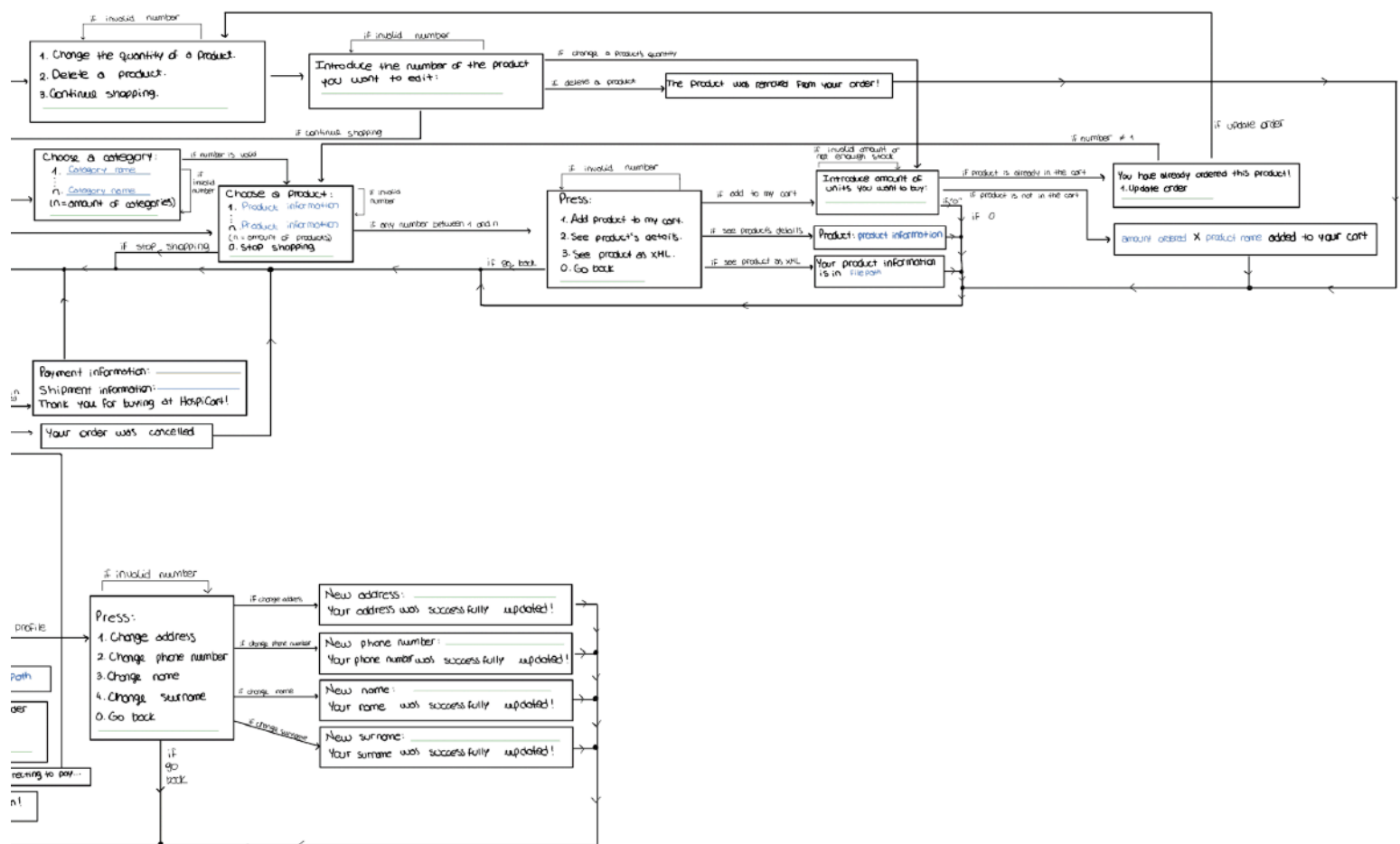


Doctor/Nurse Menu:



Closer images of the Doctor/Nurse Menu:





Supplier Menu:

